

with dimension d . We then compute our prediction as:

$$\begin{aligned}\mathbf{h} &= \text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1) \\ \mathbf{l} &= \mathbf{h}\mathbf{U} + \mathbf{b}_2 \\ \hat{\mathbf{y}} &= \text{softmax}(\mathbf{l})\end{aligned}$$

where $\mathbf{h}$ is referred to as the hidden layer, $\mathbf{l}$ is referred to as the logits, $\hat{\mathbf{y}}$ is referred to as the predictions, and $\text{ReLU}(z) = \max(z, 0)$. We will train the model to minimize cross-entropy loss:

$$J(\theta) = CE(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^3 y_j \log \hat{y}_j$$

where y_j denotes the j th element of $\mathbf{y}$. To compute the loss for the training set, we average this $J(\theta)$ across all training examples.

- Compute the derivative of $\mathbf{h} = \text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)$ with respect to $\mathbf{x}$. For simplicity, you only need to show the derivative $\frac{\partial h_i}{\partial x_j}$ for some index i and j . You may ignore the case where the derivative is not defined at 0.
- Recall in part 1b, we computed the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$. Likewise, please compute the partial derivative of $J(\theta)$ with respect to the i th entry of $\mathbf{l}$, which is denoted as l_i . Specifically, compute $\frac{\partial CE(\mathbf{y}, \hat{\mathbf{y}})}{\partial l_i}$, assuming that $\mathbf{l} \in \mathbb{R}^3$, $\hat{\mathbf{y}} \in \mathbb{R}^3$, $\mathbf{y} \in \mathbb{R}^3$, and the true label is c (i.e., $y_j = 1$ if $j = c$). **Hint:** Use the chain rule: $\frac{\partial J}{\partial l} = \frac{\partial J}{\partial \hat{\mathbf{y}}} \cdot \frac{\partial \hat{\mathbf{y}}}{\partial \mathbf{l}}$.
- We will use UAS score as our evaluation metric. UAS refers to Unlabeled Attachment Score, which is computed as the ratio between number of correctly predicted dependencies and the number of total dependencies despite of the relations (our model doesn't predict this).

In `parser_model.py` you will find skeleton code to implement this simple neural network using PyTorch. Complete the `__init__`, `embedding_lookup` and `forward` functions to implement the model. Then complete the `train_for_epoch` and `train` functions within the `run.py` file.

Finally execute `python run.py` to train your model and compute predictions on test data from Penn Treebank (annotated with Universal Dependencies).

Note:

- For this assignment, you are asked to implement Linear layer and Embedding layer. Please **DO NOT** use `torch.nn.Linear` or `torch.nn.Embedding` module in your code, otherwise you will receive deductions for this problem.
- Please follow the naming requirements in our TODO if there are any, e.g. if there are explicit requirements about variable names you have to follow them in order to receive full credits. You are free to declare other variable names if not explicitly required.

Hints:

- Each of the variables you are asked to declare (`self.embed_to_hidden_weight`, `self.embed_to_hidden_bias`, `self.hidden_to_logits_weight`, `self.hidden_to_logits_bias`) corresponds to one of the variables above ($\mathbf{W}$, $\mathbf{b}_1$, $\mathbf{U}$, $\mathbf{b}_2$).
- It may help to work backwards in the algorithm (start from $\hat{\mathbf{y}}$) and keep track of the matrix/vector sizes.
- Once you have implemented `embedding_lookup` (e) or `forward` (f) you can call `python parser_model.py` with flag `-e` or `-f` or both to run sanity checks with each

function. These sanity checks are fairly basic and passing them doesn't mean your code is bug free.

- When debugging, you can add a debug flag: `python run.py -d`. This will cause the code to run over a small subset of the data, so that training the model won't take as long. Make sure to remove the `-d` flag to run the full model once you are done debugging.
- When running with debug mode, you should be able to get a loss smaller than 0.2 and a UAS larger than 65 on the dev set (although in rare cases your results may be lower, there is some randomness when training).
- It should take up to **15 minutes** to train the model on the entire training dataset, i.e., when debug mode is disabled.
- When debug mode is disabled, you should be able to get a loss smaller than 0.08 on the train set and an Unlabeled Attachment Score larger than 87 on the dev set. For comparison, the model in the original neural dependency parsing paper gets 92.5 UAS. If you want, you can tweak the hyperparameters for your model (hidden layer size, hyperparameters for Adam, number of epochs, etc.) to improve the performance (but you are not required to do so).

Deliverables:

- Working implementation of the transition mechanics that the neural dependency parser uses in `parser_transitions.py`.
- Working implementation of minibatch dependency parsing in `parser_transitions.py`.
- Working implementation of the neural dependency parser in `parser_model.py`. (We'll look at and run this code for grading).
- Working implementation of the functions for training in `run.py`. (We'll look at and run this code for grading).
- Please use efficient functions and **avoid for loops** when implementing `embedding_lookup`. **Otherwise, you may exceed the GradeScope test time limit.**
- **Report the best UAS your model achieves on the dev set and the UAS it achieves on the test set in your written submission.** You can report it in the PDF and tag the page.

Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for Assignment 2 [coding] and another for Assignment 2 [written]:

1. Run the `collect_submission.sh` script to produce your `assignment2.zip` file.
2. Upload your `assignment2.zip` file to GradeScope to Assignment 2 [coding].
3. Upload your written solutions to GradeScope to Assignment 2 [written].